

Introduction

This project that I have developed as part of my “Maths for computing” module was focused on using mathematical concepts such as forces, vectors, gravity and more to create an interactive simulation where various different forces are visualised using my programming code created in p5.js. The theme i had chosen to represent my work in this project was an ocean scene featuring a boat that reacts to wind and waves, with the additional presence of rain particles that are affected by gravity. The project i have created was made using javascript and the p5.js library which offers good tools that helped me across this project to find new ways of adding forces and elements into my interactive project.

The purpose of my simulation was to demonstrate how different forces such as wind, gravity and wave motion can be modeled using trigonometry functions and vector mathematics. Each aspect of the simulation including wave generating, boat movement and rain behaviour were made to show an attempt at a real life scenario in a fun and interactive project. Another important feature added to my project is a directional arrow that appears when the user applies wind force. This visual element in my project allows the user to see exactly which way the force is being applied, making the simulation's physics more understandable. Overall my project combines programming logic, interactive design and mathematics to bring this project together. It showcases my technical skills and my new skills of combining math functions into my programming to create digital work.

Project goals/concept

The core goal of this project was to apply mathematical principles in a computing way to overall simulate the natural motions and forces that may happen in my given scene that I created and to display this using my coding knowledge in an interactive way. I started to create my simulation where a boat floats on the animated **sine** waves in which i created using trigonometric functions in p5.js i then added the idea of wind in my scene which is a force i think fits well into this scenario and also finally rain in which falls on command of a key being pressed that can also display its own force of gravity and shows in my project how the rain particles can be affected by the wind. These three main attributes to my project (waves, wind and rain) each represent a different branch of maths such as trigonometry for the waves, vectors for the wind and acceleration and motion equations for the rain.

This overall concept I designed in my head was intended to be visually appealing while focusing on a few core mechanics that can be explored and used in detail rather than cluttering the scene with too many features. The boat responds dynamically to the users interaction, the rain particles that I added respond to both the wind forces and gravity and the waves are animated using specific **sine** functions. The inclusion i added of a red wind direction arrow was key to making my interaction more intuitive for my users showing them exactly how their actions were influencing the system on the screen.

Basic Goal & How do I propose to accomplish it

The projects basic objective was to model three simple core physical systems (ocean waves, wind force and gravity) that worked together to display a life like scenario of forces shown in my creative and mathematical aspect. I intended to accomplish this in my project by designing and implementing three different classes that work together to form my interactive project, these being my main `JS sketch.js` class which holds most of the important code including the boat formation and wave formation, my `JS Wind.js` class which holds the wind force system using vectors and mathematics and finally my `JS Rain.js` class which holds the raining system based on velocity and acceleration principles when the “R” key is held down.

I planned to develop each part of this simulation individually, testing and refining each class before integrating them together as a whole. For example, I first created the wave system using **sine** functions to simulate the natural motions of the water, however also implementing maths into this to show my learning. Then a wind class was developed that I made to use mouse input to calculate direction vectors and apply force. The rain system was something that i added last using gravitational acceleration to move particles downward and applying wind to simulate the particles moving in the wind.

The cartoon styled boat i used itself needed to not just move in a boring unrealistic way but instead float along the waves in a realistic way. This required syncing the boats centre position with the dynamic wave surface. A final addition was the wind direction arrow which appears onto the screen whenever the wind force is active. This arrow provides immediate visual feedback, making the simulation understandable for the users.

Functions and my process (Math)

Wave Generation Using Trigonometry

The wave motion that i added is generated using sine functions which create me a natural oscillating pattern ideal for simulating ocean waves. The y coordinate of each of my waves is calculated with the sine function `let y = amplitude * sin(frequency * (x + Offset));` in this function shown that i used, the value amplitude is used to control the waves height, frequency is used to control the determine the number of wave cycles across the screen and offset is used to add a horizontal shift to animate the wave and give the illusion of motion.

To enhance the visual depth of these cosine waves I added three different **sine** waves with slightly different values for frequency, amplitude and speed. These different layered waves move at slightly different rates to create a more realistic ocean like effect where not 1 layer moves exactly the same. Each wave is also drawn with transparency to blend together so the effects can be seen clearly.

The boat's vertical position is also constantly updated based on the sine value at its current x position by using a loop that checks if the current x coordinate equals the boat's x coordinate as well, the boat is placed exactly at the wave height underneath it. This ensures that as the wave rises and falls, the boat appears to float naturally across the sine wave left and right when the wind pushes it the x coordinate is increased

Wind System Using Vectors

The wind system in my project is based on vector mathematics, using p5.js's vector class to manage direction and magnitude. When the user clicks the mouse, the direction of the wind is calculated by creating a vector from the mouse's position to the boat's current position. This vector is then normalized which overall means it is reduced to a certain length to save the direction without affecting the speed and it is multiplied by a constant magnitude to determine the overall force that is applied.

The cartoon boat images position is adjusted each frame by this force which allows the boat to drift left or right on screen floating on the cosine waves i created depending on the users mouse location affecting the wind direction. To attempt to keep my boat going off of the screen too much I added a constraint function which I applied to my code. The system I created realistically models how wind might influence a floating object that is portrayed on a bed of moving waves with a consistent and controllable method of input.

One particularly useful feature which I added late on in my project was the use of a directional arrow drawn only when the wind force is active. This arrow uses the same force vector and draws a line which extends from the user's mouse input to the boat in the direction of the force that is applied visually representing both the direction and the approximate strength of the wind being applied.

Arrow for visualising wind

To help my users understand how my wind force vector affects my boat I implemented at the last minute a directional arrow. This arrow only appears when the wind force is active and when the user is holding down the left click mouse button which is what activates the wind force. The arrow I created is drawn using the same vector that is used to apply the wind force to the boat and is multiplied by a scalar to make the arrow visually long enough to be clear.

The arrow i implemented starts from the users mouse position and extends outwards in the direction of the force, showing the direction that the boat will be pushed or the direction in which the rain will be pushed from, this addition enhances the usability of my project and overall helps bridge the gap between abstract vector math and the users interaction showing the educational side of my simulation.

Rain Particle system

One feature in my simulation that i think is the more dynamic and visually engaging feature in my project is the rain particle system which not only just adds to the realistic element i was intending on going for but also serves as a functional representation of how gravity and wind forces can affect particles differently in p5.js. I made it so that the rain is triggered by pressing the 'r' key which causes new rain particles to spawn at random positions across the width of the canvas. With vertical starting positions implemented at the top of the screen above the canvas this simulates rainfall beginning from high in the sky rather than starting at a random point in the canvas.

The logic behind each rain particle is held within the RainParticle class. When the user has pressed the 'r' key and the class is initiated each particle is given a random x-position and a random y-position that is visible above the canvas `this.y = random(-100, -10);`, a

random vertical speed was also given `this.speed = random(4, 10);` this is to ensure the simulation stayed realistic as it looks unrealistic when each raindrop falls at the same speed, there is also a random length that represents the rain particles sizes. Each particle has a velocity vector initialised to move downward. This is represented in my code with

`this.velocity = createVector(0, this.speed);`. In the update method the particles position is modified by adding its velocity up to its current position as shown in this code that i

implemented `this.x += this.velocity.x; this.y += this.velocity.y;` this simple however effect method that i used simulates the gravity by keeping the motion consistently downward unless acted on by another force.

To integrate this rain particle system with other forces i began to add wind force interaction with each particle. To do this in my code i used the `.applyWindForce(scaledWindForce);` function to ensure that my vector would effect the rain realistically and to add a scaled version of the winds vector to the particles velocity. The scaling i added here is crucial as while the boat may respond more strongly to the wind the rain will only respond slightly which overall mimics realistically the drift of rainfall in the air when being pushed by a wind force. This is all implemented in the main draw loop with the code:

`let scaledWindForce = windForce.copy().mult(0.2);` and

`rain[i].applyWindForce(scaledWindForce);` By applying the wind in the way that i did the particles change direction only slightly as they fall which makes the rain look more natural and dynamic which also portrays in my simulation a great example of varying force magnitude depending on what an object's properties are. In addition to this my programming

code includes an `offScreen()` method to check if a particle has exceeded the canvas and if this case is true then the particle will be removed from the array to overall optimize the

performance of the simulation. `if (rain[i].offScreen())
rain.splice(i, 1);`

Wind Direction Visualisation

To improve my users interaction with my simulation and the feedback in which they give i implemented a visual indicator for wind direction using an arrow that appears whenever the wind force is active as previously stated before. This helps my users to clearly see which direction that the wind is affecting the boat and rain. I drew the arrow in my code as a stand out red colour using `stroke(255, 0, 0);` and then positioned this near the top of the canvas in my open space using `(height * 0.1);`. The arrow i made in constructed using a base vector and then extends in the direction of the wind vector scaled for visibility

```
(windForce.copy().mult(50));
```

How Successful Was The Implementation

Overall my implementation of forces in my simulation was highly successful. The cosine waves visually simulate the motion of the ocean very well, while the boat reacts to the wind input in a realistic and dynamic way and also reacts to the cosine waves in a dynamic way. The rain system that was implemented integrates gravity and wind very well and the inclusion of the visualisation wind direction arrow that I added provides real time visual feedback that reinforce the concepts of the forces and vector direction.

The math involved behind the cosine waves involving amplitude, frequency and offset work well in creating layered and animated wave visuals. Furthermore the use of vector mathematics throughout the p5.js library helped make the whole implementation of the direction based wind forces both efficient and realistic as this gave the whole project a strong foundation in my physics based animation while still remaining visually engaging .

Problems I Faced & How I overcame them

Several different challenges did emerge during the development of my project . One major issue that I came across was syncing the boat's position with the wave height. Originally, the boat would float unevenly or appear disconnected from the wave which made the simulation overall look unrealistic and very unprofessional. The fix I figured out was to use a loop that matched the boats x position into the exact x value that is in the wave loop and then in turn assign the boats y coordinate based on that y value given.

Another challenge that I came through was controlling the overall wind direction. At first applying the wind from the mouse position to the boat's position caused the boat to move towards the mouse's position which was the opposite of what i was wanting. To correct this error that occurred, reversing the vector using subtraction and normalization fixed this error and made it so that the boat moved away from the user's cursor and applied the correct wind force that was intended.

At one point in my simulation performance through vs code and the browser that i would be running the simulation on became a concern when the rain particles were accumulating too quickly. Initially , articles were not being removed when they moved off screen. This caused the frame rate of my simulation to drop and i was stuck with solving this for a long period of my project, with the help of the p5.js library and teachers help this was solved by checking their y positions and using `array.splice(i, 1);` to remove them from the array once they had fallen off of the screen.

Implementing the wind arrow was simple in concept when I was implementing this into my code however it became difficult to scale properly. If it was too short then the users couldn't see it and then again if it was too long it looked cartoon-ish and took away what the main use of my simulation was for as it was taking up most of the screen. After some trial and error I scaled the vector to a more applicable length that was clearly visible without taking up most of the scene. However this problem that i faced was not so much of a worry however was crucial to fix.

How successful did this turn out

I think that my project turned out to be both visually appealing and looking quite simple but effective while also being technically accurate and educational too. Each system that was integrated (wind, waves and rain) functioned as expected and all integrated smoothly with each other. The boat I think moves naturally in my simulation on top of cosine waves and the waves themselves have depth and have a layered fluidity being that there is 3 of them that move in different ways and then the rain rects realistically with both the gravity and wind.

The simulation I produce I think achieves its educational goals that I set out to aim as well. Through the interaction of my project my users can observe how vector forces work, how sine waves can generate motion and how gravity obviously influences falling objects all through coding in p5.js. The arrow I also added does really visualize wind direction too which I think provides feedback and user understanding to help.

While I do think my project could be expanded to include more complex forces that could also work together such as buoyancy or friction/drag, I think this simulation does serve as a complete model even though it is simplistic. I think that it balances realism with the cartoon-like effect I added making it fun and engaging, however also educational.

Evaluation

In my opinion my project met its initial goals that I set out for. I set out to create an interactive simulation where forces such as wind and gravity could be observed through simple inputs and also visual feedback. The end result of my simulation project showcases vector mathematics, forces working together in real time being implemented in p5.js and the structure of my code easily allows for expansion possibly in the future if i decide to proceed with this simulation as i do think other things could be useful to add into this like said before such as buoyancy and other forces.

The choice of forces that I picked (wind for directional control and standard gravity for rain) were carefully selected to demonstrate the varying interactions on different elements. The maths-like approach to creating the waves motion using the cosine functions added some complexity to the waves while it also improved the visual quality of the scene.

What I think makes this project particularly effective is its integration of interactivity that it has and the visual feedback and physics that are implemented. I think that the use of the user being able to manipulate the wind, rain and boat in whatever way they want to provides a consistent application of forces to multiple systems and makes my project more enjoyable to use and the inclusion of the wind direction arrow brings clarity and immediate understanding of how vectors influence movement in a 2D environment.